IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Lucas Lopatka
October 26, 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

Summary of methodologies

- Collected launch data by scraping the Wikipedia launch tables (the SpaceX API was attempted, but it was unavailable)
- Wrangled the course dataset of 90 Falcon 9 launches and created a 0/1 landing label (Class)
- Explored it with SQL, Seaborn charts, Folium maps and a Plotly Dash dashboard
- Trained 4 classifiers (LogReg, SVM, Decision Tree, KNN), tuned with GridSearchCV (10-fold)

Summary of all results

- 67% of first stages landed overall; the yearly rate rose from 0% (2013) to ~85–90% (2019–2020)
- Orbit, payload and launch site all relate to success: KSC LC-39A has the best record (10 of 13)
- Logistic Regression, SVM and KNN reach 83.3% test accuracy; the tree reaches 77.8%
- No model misses a real landing, but all over-predict success

Introduction

Project background and context

- SpaceX advertises Falcon 9 launches at about $62M; other providers charge $165M or more
- Most of the savings come from landing and reusing the first stage
- If we can predict a landing, we can estimate launch cost, which helps a competitor bid against SpaceX
- Data: Wikipedia launch records and the course launch dataset, 2010–2020

Problems we want to answer

- Which factors drive landing success: payload, orbit, launch site, experience?
- How did the success rate change over time?
- Where are the launch sites, and what is around them?
- Which classification model predicts landings best?

Section 1

Methodology



Methodology

1	**Data collection**	Web scraping of the Wikipedia launch tables (SpaceX API attempted)
2	**Data wrangling**	Check missing values, group landing outcomes, create the 0/1 Class label
3	**EDA with visualization and SQL**	Seaborn charts and 10 SQL queries on the launch database
4	**Interactive visual analytics**	Folium maps (sites, outcomes, distances) and a Plotly Dash dashboard
5	**Predictive analysis**	Build, tune (GridSearchCV) and evaluate 4 classification models

Data Collection



Web scraping (completed)

- requests + BeautifulSoup on a fixed Wikipedia revision (reproducible)
- Looped through the launch tables row by row with helper functions
- Extracted flight no., date/time, booster, site, payload, mass, orbit, customer, outcome and landing

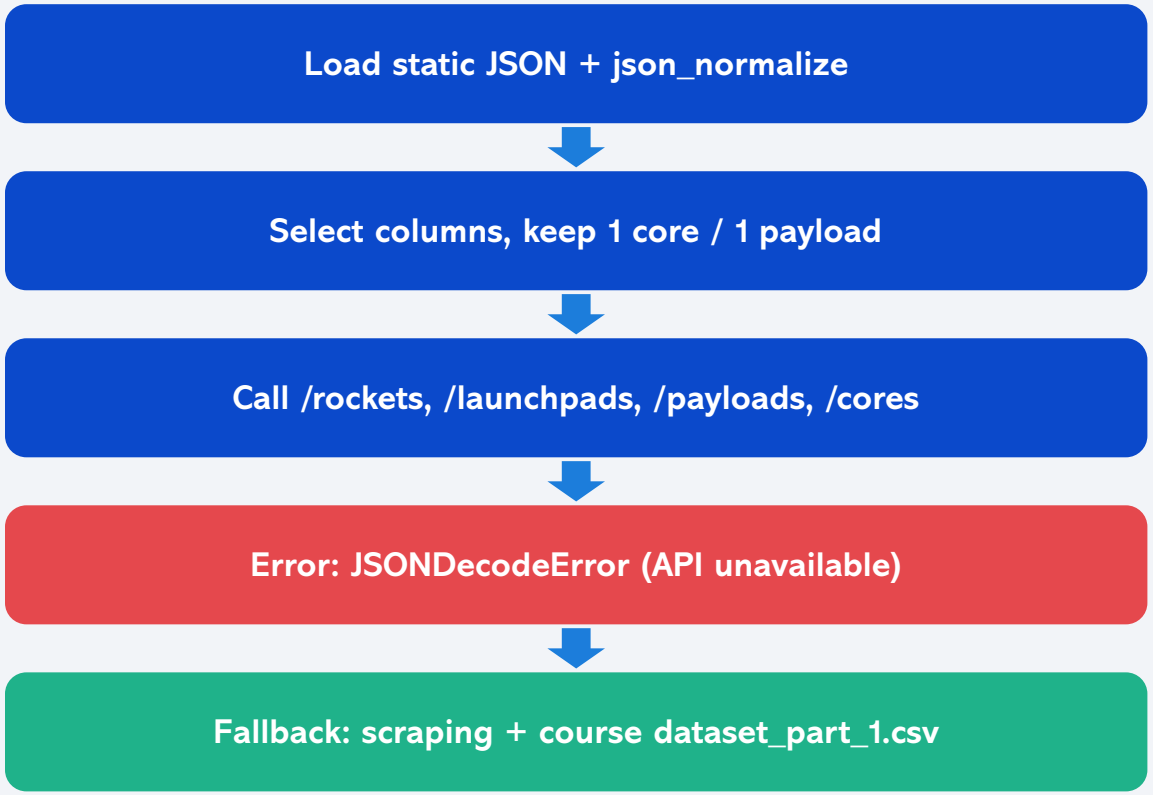
SpaceX REST API (attempted)

- Planned calls to /rockets, /launchpads, /payloads and /cores
- The requests failed with JSONDecodeError (empty, non-JSON reply)
- Fallback: the course's pre-built dataset_part_1.csv, which has the same fields

Data Collection – SpaceX API (Attempted)

What happened

- Planned: static JSON + calls to /rockets, /launchpads, /payloads and /cores
- Result: the requests returned empty or non-JSON replies (JSONDecodeError)
- Reason: the community-run API was unavailable from the lab
- Fallback: web scraping + the course's dataset_part_1.csv

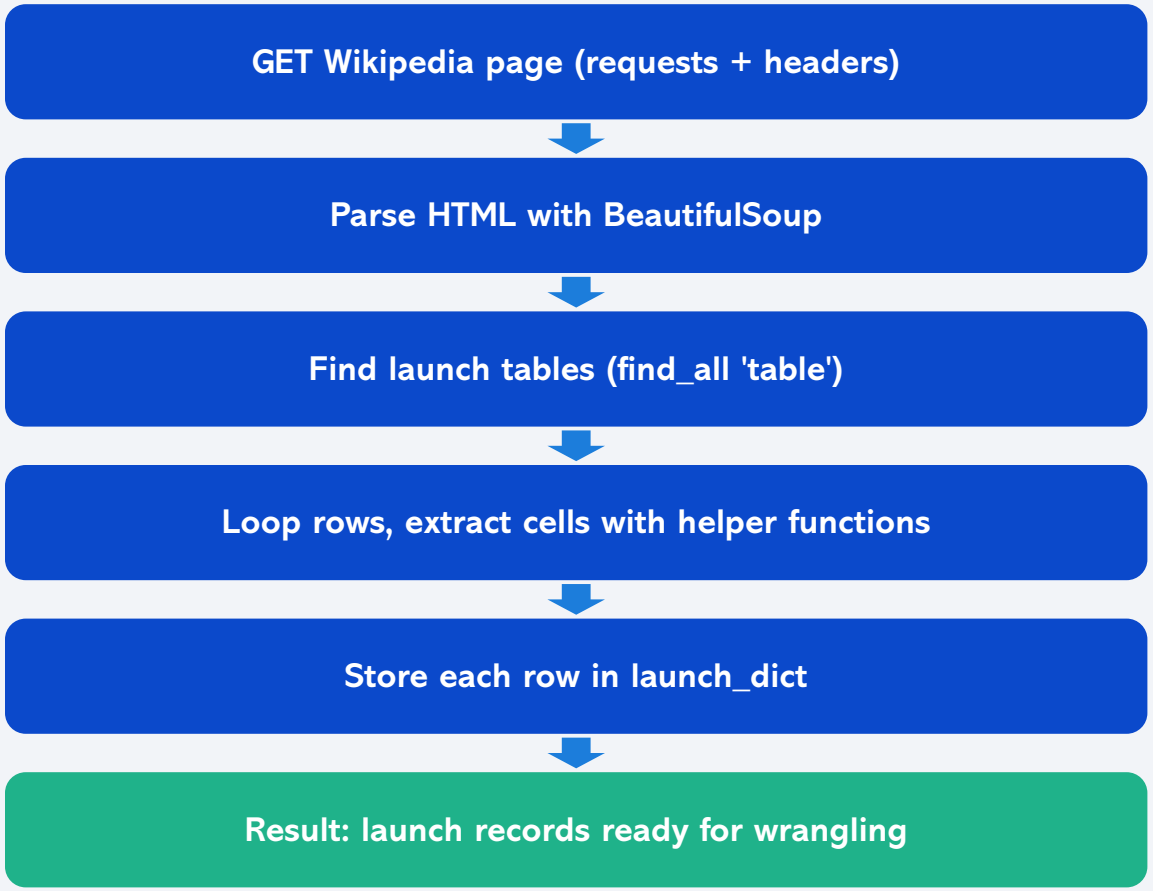


Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_1.ipynb

Data Collection – Scraping

Key phrases

- HTTP GET with a browser User-Agent on a fixed Wikipedia revision
- BeautifulSoup parses the launch tables; helper functions clean date, booster, mass and landing cells
- Each row is stored in launch_dict for later wrangling



Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_1.ipynb

Data Wrangling

Load course dataset_part_1.csv

↓

Check missing values and data types

↓

Count launches per site, orbit and outcome

↓

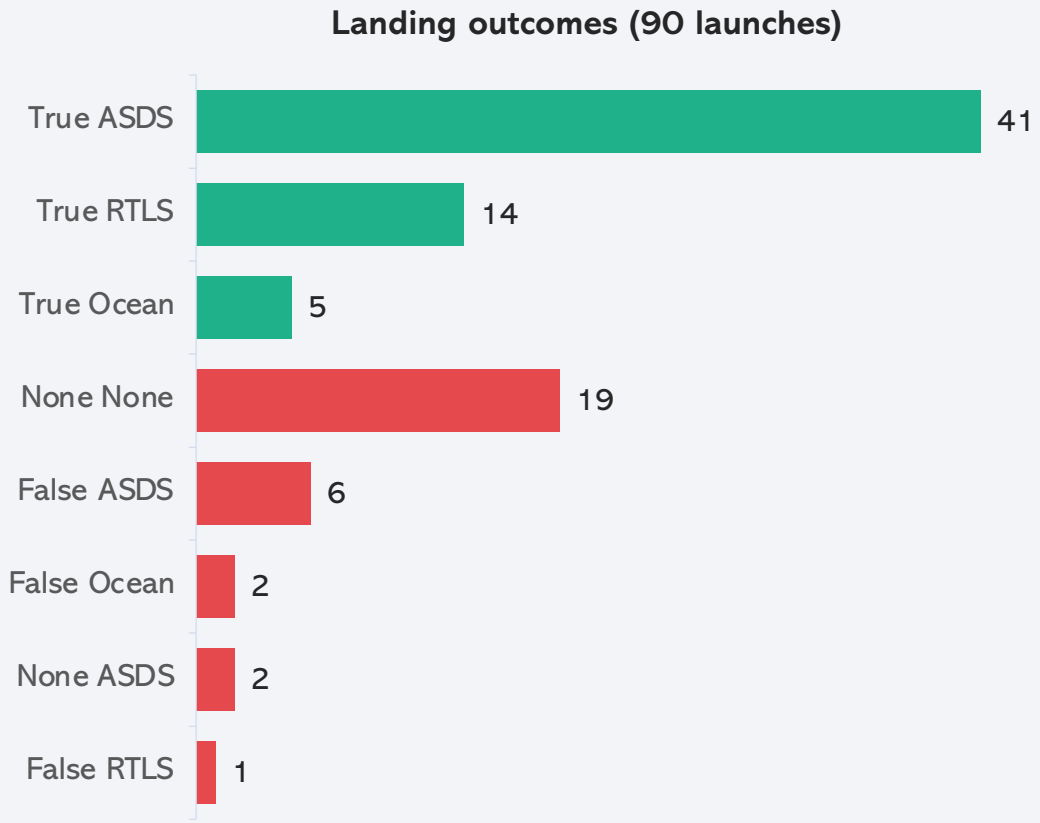
Define 'bad' outcomes (did not land)

↓

Create Class (1 = landed, 0 = did not)

↓

Save dataset_part_2.csv



60 landed vs 30 did not: success rate 66.7%

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_1.ipynb

EDA with Data Visualization

Chart	Plot type	Why we used it
Flight Number vs Payload Mass	catplot	See whether experience and payload relate to landing success
Flight Number vs Launch Site	catplot	Check how each site was used over time
Payload Mass vs Launch Site	catplot	Find payload limits and outcomes per site
Success rate by Orbit	barplot	Compare orbit types at a glance
Flight Number / Payload vs Orbit	scatterplot	Spot orbit-specific patterns
Yearly success rate	lineplot	Show the trend over time

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

EDA with SQL

Queries 1–5

- Unique launch site names
- 5 records where the site starts with 'CCA'
- Total payload mass for NASA (CRS)
- Average payload mass of booster F9 v1.1
- Date of the first successful ground-pad landing

Queries 6–10

- Boosters with drone-ship success and payload between 4,000 and 6,000 kg
- Total successful and failed mission outcomes
- Boosters that carried the maximum payload
- 2015 drone-ship landing failures with booster and site
- Ranking of landing outcomes between 2010-06-04 and 2017-03-20

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

Build an Interactive Map with Folium

Map object	What we added	Why
Circle + Popup	Highlighted circle at each launch site	Mark exact site locations
Marker (DivIcon)	Text label with the site name	Identify each site on the map
MarkerCluster	Green / red markers for each launch	Show landing success and failure per site
MousePosition	Live latitude / longitude readout	Find coordinates of coast, road, rail, city
Marker + PolyLine	Lines with distances (Haversine, km)	Measure proximity to coast, highway, railway, city

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_3.ipynb

Build a Dashboard with Plotly Dash

Component	Interaction	Why
Launch-site dropdown	All sites or one site	Let users focus on a single site
Pie chart (callback)	Success share by site, or success vs failure for one site	Compare sites and see each site's success ratio
Payload range slider	Filter payload from 0 to 10,000 kg	Test how payload relates to success
Scatter chart (callback)	Payload vs outcome, coloured by booster version	Find which payloads and boosters succeed most

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_3.ipynb

Predictive Analysis (Classification)



Models and tuning

- Logistic Regression: C, penalty, solver
- SVM: kernel, C, gamma
- Decision Tree: criterion, depth, leaf and split sizes
- KNN: n_neighbors, algorithm, p

Best performing model

- Logistic Regression, SVM and KNN tie at 83.3% test accuracy
- Decision Tree has the best CV score (86.3%) but the lowest test accuracy (77.8%)
- Small test set: one prediction = 5.6 points

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_4.ipynb

Results

EDA results

- Success grows with experience: 0% in 2013, ~85–90% in 2019–2020
- Orbit and payload matter: GTO is hardest, light SSO/VLEO missions do well
- KSC LC-39A has the best record

Interactive analytics

- Folium: 4 sites, all on the coast, far from cities
- Distances: coast 0.9 km, highway 0.6 km, railway 1.3 km, city 23 km
- Dash: success by site and payload range

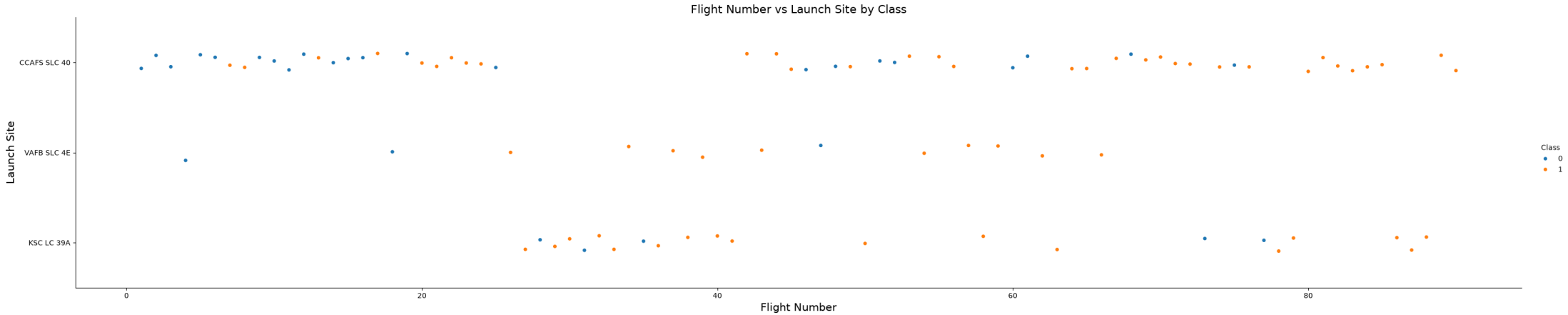
Predictive analysis

- 4 tuned classifiers
- LogReg, SVM, KNN: 83.3% test accuracy
- No missed landings, 3 false alarms out of 6 non-landings

Section 2

Insights drawn from EDA

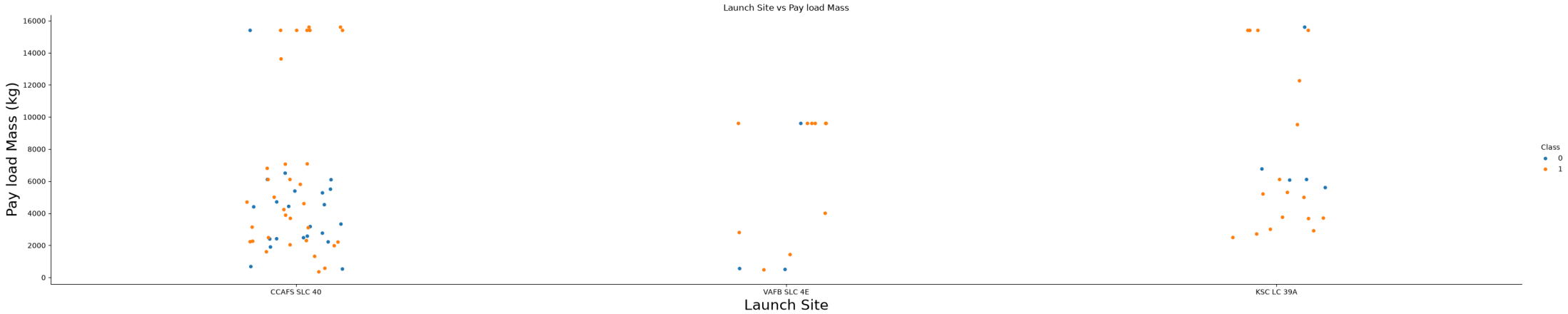
Flight Number vs. Launch Site



What we see

- CCAFS SLC 40 is used throughout the program (55 of 90 launches); VAFB SLC 4E has 13 and KSC LC 39A has 22
- KSC LC 39A only appears from about flight 27
- Early flights (blue, did not land) dominate flights 1–25; later flights (orange) mostly land

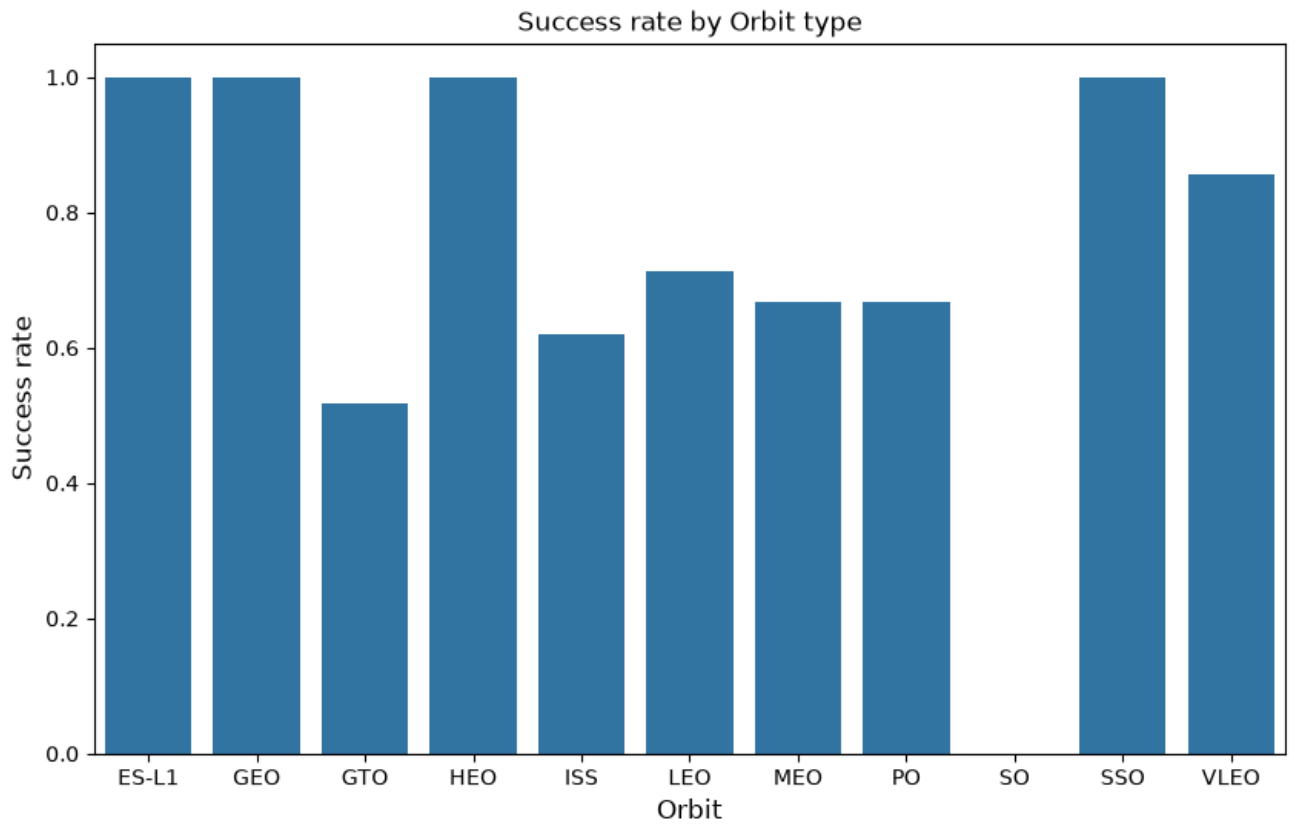
Payload vs. Launch Site



What we see

- VAFB SLC 4E never launched payloads above about 10,000 kg
- The heaviest payloads (~15,000 kg) fly from CCAFS and KSC, and most of them land
- Light and mid payloads (below 7,000 kg) show both outcomes, so payload alone does not decide success

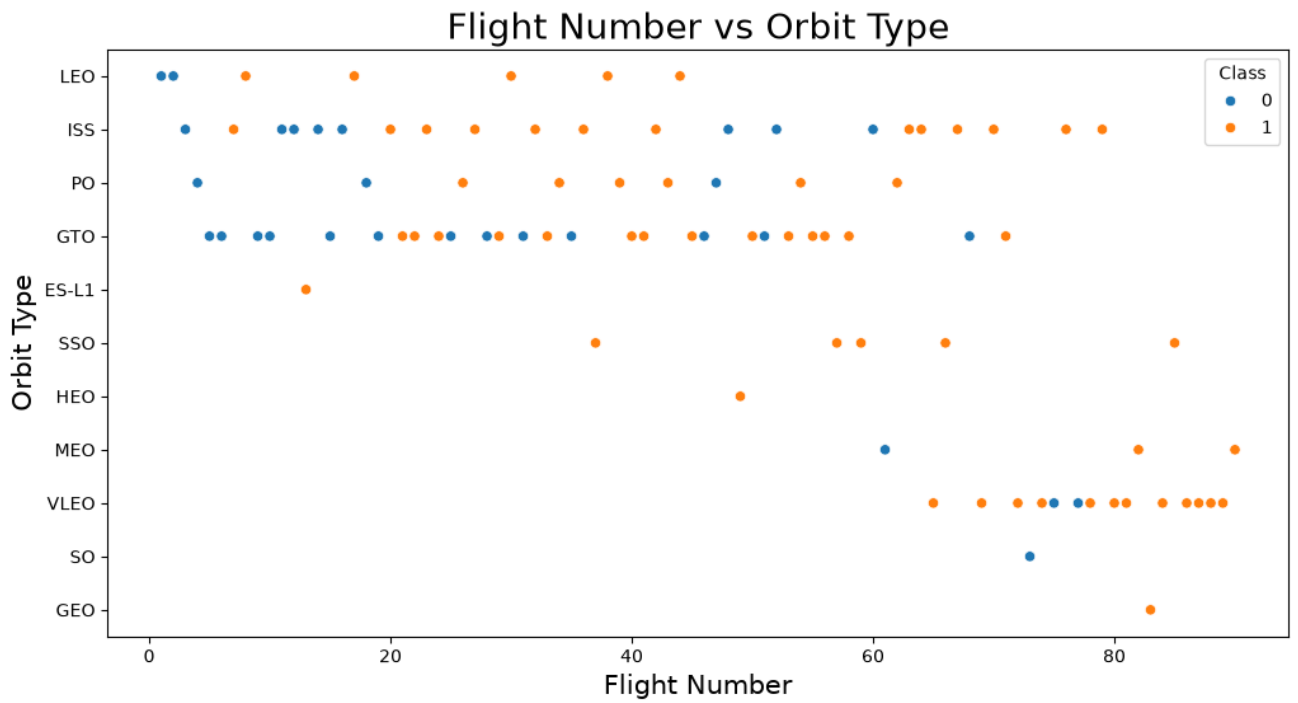
Success Rate vs. Orbit Type



What we see

- ES-L1, GEO, HEO and SSO show 100% success, but with only 1 to 5 launches each
- VLEO is about 86%
- GTO (about 52%) and ISS (about 62%) are the large groups with the weakest results
- SO has 0% (a single launch)

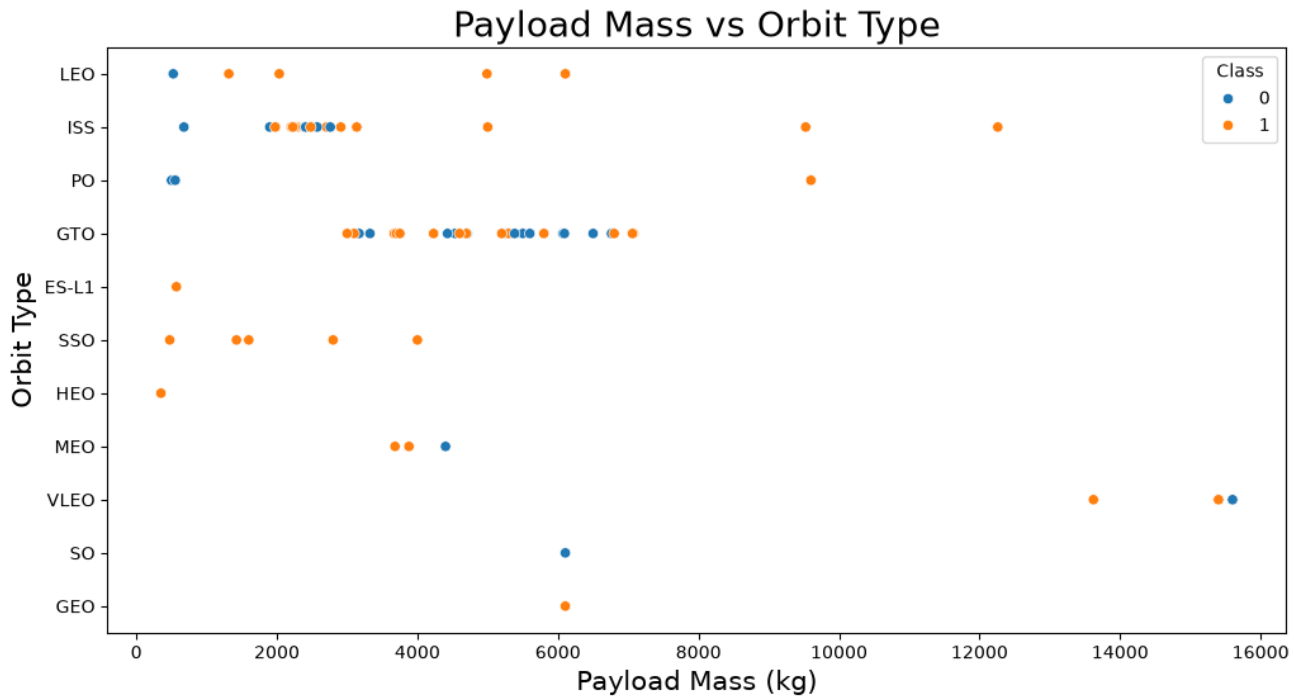
Flight Number vs. Orbit Type



What we see

- In LEO, early flights failed and later ones landed
- VLEO flights only appear late in the program and mostly land
- GTO shows both outcomes across many flights, with no clear trend

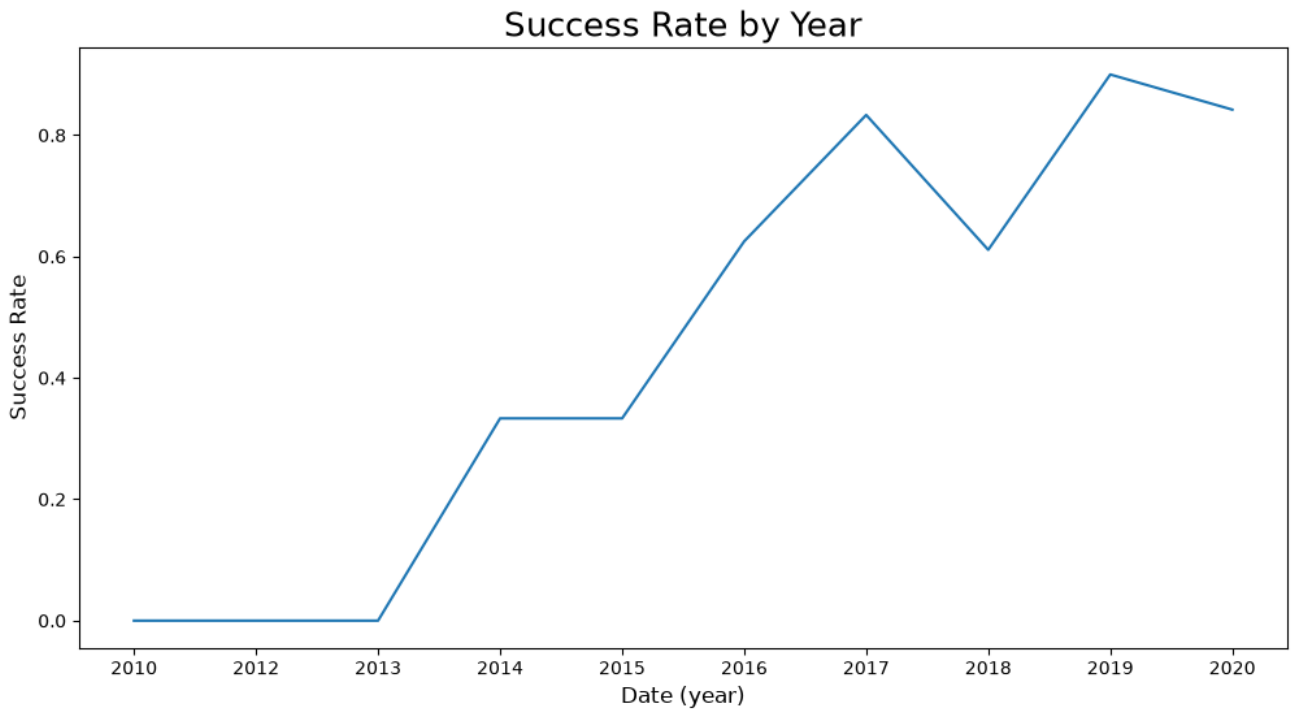
Payload vs. Orbit Type



What we see

- Payloads above 9,000 kg appear in ISS, PO and VLEO missions and mostly land
- GTO payloads cluster between 3,000 and 7,000 kg, with mixed outcomes
- SSO, HEO and ES-L1 missions are light and successful, but few

Launch Success Yearly Trend



What we see

- 0% success until 2013
- Rate climbs from 2014 and reaches about 60% in 2016
- Around 83% in 2017, a dip in 2018, then about 90% in 2019 and 84% in 2020

All Launch Site Names

```
SELECT DISTINCT Launch_Site
FROM SPACEXTBL;
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Explanation

- DISTINCT removes duplicates
- The data holds 4 site names: two at Cape Canaveral (LC-40 and SLC-40, the renamed pad), one at Kennedy and one at Vandenberg

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

Launch Site Names Begin with 'CCA'

```
SELECT * FROM SPACEXTBL
WHERE Launch_Site LIKE 'CCA%'
LIMIT 5;
```

Date	Booster_Version	Launch_Site	Payload	Mass (kg)	Orbit
2010-06-04	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO
2010-12-08	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats	0	LEO (ISS)
2012-05-22	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)
2012-10-08	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)
2013-03-01	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)

LIKE 'CCA%' keeps sites starting with CCA. These are the first five Falcon 9 v1.0 flights, all from CCAFS LC-40.

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

Total Payload Mass

```
SELECT SUM(PAYLOAD_MASS__KG_) AS TOTAL_PAYLOAD_MASS_KG
FROM SPACEXTBL
WHERE Customer = 'NASA (CRS)';
```

45,596 kg

total payload for NASA (CRS)

Explanation

- SUM adds the payload mass of all NASA (CRS) missions
- CRS = Commercial Resupply Services flights to the ISS

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

Average Payload Mass by F9 v1.1

```
SELECT AVG(PAYLOAD_MASS__KG_) AS AVG_PAYLOAD_MASS_KG
FROM SPACEXTBL
WHERE Booster_Version = 'F9 v1.1';
```

2,928 kg

average payload of booster F9 v1.1 (2,928.4)

Explanation

- AVG over all rows with booster version exactly 'F9 v1.1'
- A mid-size payload compared with the Block 5 boosters (about 15,000 kg)

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

First Successful Ground Landing Date

```
SELECT MIN(DATE) AS DATE
FROM SPACEXTBL
WHERE Landing_Outcome = 'Success (ground pad)';
```

2015-12-22

first successful landing on a ground pad

Explanation

- MIN(DATE) returns the earliest date that matches
- Ground-pad landings (RTLS) began in December 2015, after the first drone-ship attempts in early 2015

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

Drone Ship Landings, Payload 4000–6000 kg

```
SELECT Booster_Version FROM SPACEXTBL
WHERE PAYLOAD_MASS__KG_ > 4000
  AND PAYLOAD_MASS__KG_ < 6000
  AND Landing_Outcome = 'Success (drone ship)';
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

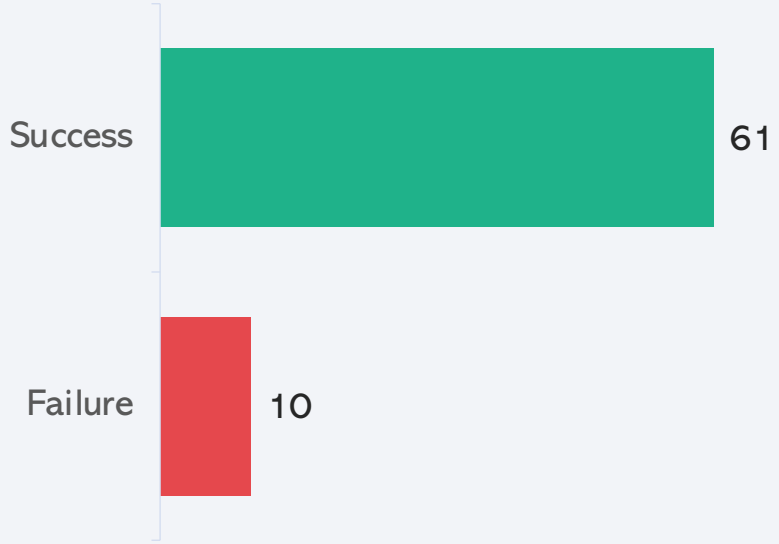
Explanation

- Four boosters landed on a drone ship with a mid-range payload
- All are Falcon 9 Full Thrust (FT) boosters; B1021.2 and B1031.2 are reused boosters (second flight)

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

Total Number of Successful and Failure Mission Outcomes

```
SELECT COUNT(Date), CASE WHEN Landing_Outcome LIKE '%Success%' THEN 'Success'
       WHEN Landing_Outcome LIKE '%Failure%' THEN 'Failure' END AS Grouped
FROM SPACEXTBL WHERE Landing_Outcome LIKE '%Failure%'
   OR Landing_Outcome LIKE '%Success%' GROUP BY Grouped;
```



Explanation

- 61 successful and 10 failed landing outcomes
- CASE WHEN groups the detailed outcomes (drone ship, ground pad, parachute) into Success or Failure; 'No attempt' rows are excluded

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

Boosters Carried Maximum Payload

```
SELECT DISTINCT Booster_Version FROM SPACEXTBL
WHERE PAYLOAD_MASS__KG_ =
      (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTBL);
```

Booster_Version			
F9 B5 B1048.4	F9 B5 B1049.4	F9 B5 B1051.3	F9 B5 B1056.4
F9 B5 B1048.5	F9 B5 B1051.4	F9 B5 B1049.5	F9 B5 B1060.2
F9 B5 B1058.3	F9 B5 B1051.6	F9 B5 B1060.3	F9 B5 B1049.7

- The subquery finds the maximum payload mass, then lists every booster that carried it.
- Twelve Block 5 (B5) boosters share the record payload, several on their 2nd to 7th flight.

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

2015 Launch Records

```
SELECT SUBSTR(DATE,6,2) AS MONTH, SUBSTR(DATE,0,5) AS YEAR,
       Booster_Version, Landing_Outcome, Launch_Site
FROM SPACEXTBL
WHERE Landing_Outcome = 'Failure (drone ship)'
  AND SUBSTR(DATE,0,5) = '2015';
```

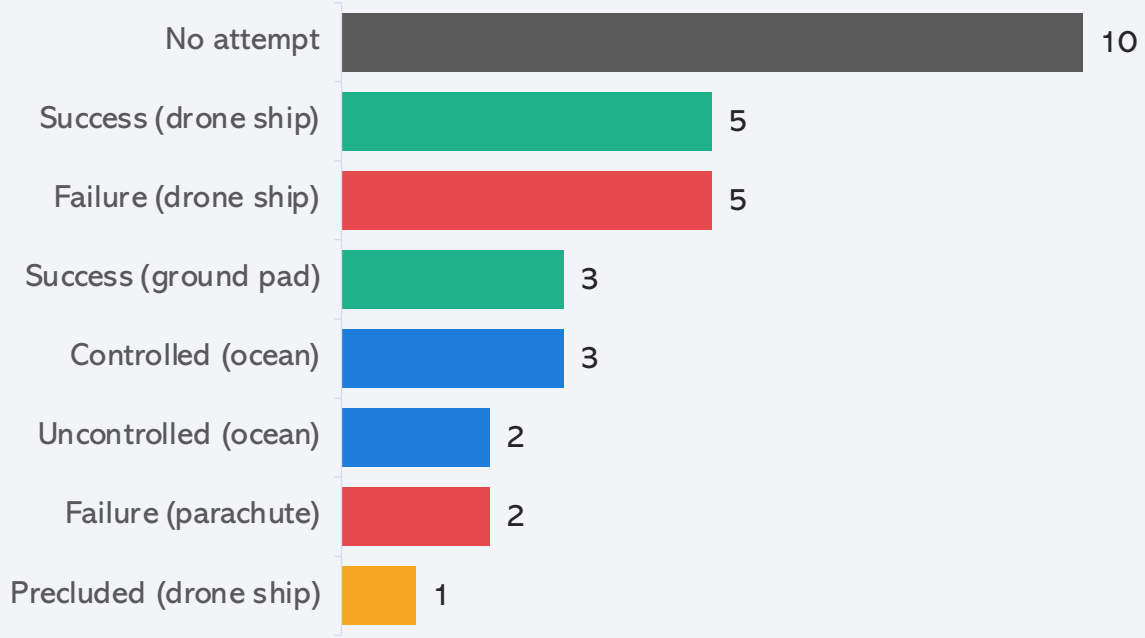
MONTH	YEAR	Booster_Version	Landing_Outcome	Launch_Site
01	2015	F9 v1.1 B1012	Failure (drone ship)	CCAFS LC-40
04	2015	F9 v1.1 B1015	Failure (drone ship)	CCAFS LC-40

- SUBSTR extracts the month and year from the date text
- Both 2015 drone-ship failures were F9 v1.1 boosters launched from CCAFS LC-40 (January and April)

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb

Rank Landing Outcomes, 2010-06-04 to 2017-03-20

```
SELECT COUNT(Date) AS Count, Landing_Outcome FROM SPACEXTBL
WHERE DATE BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY Landing_Outcome ORDER BY Count DESC;
```



Explanation

- Most early launches did not try to land (10)
- Drone-ship attempts split evenly: 5 successes and 5 failures
- Ground-pad landings: 3 successes

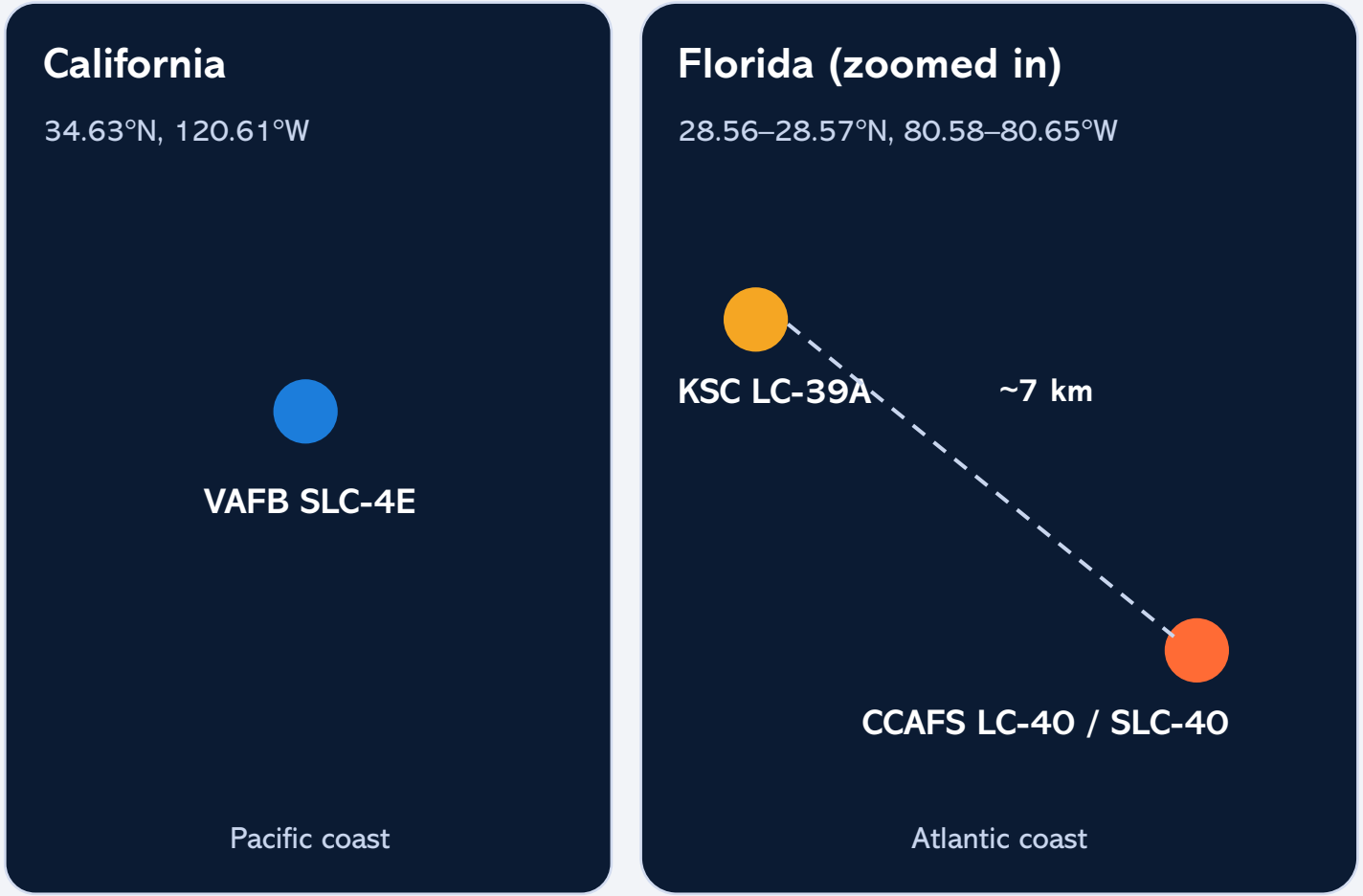
Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_2.ipynb



Section 3

Launch Sites Proximities Analysis

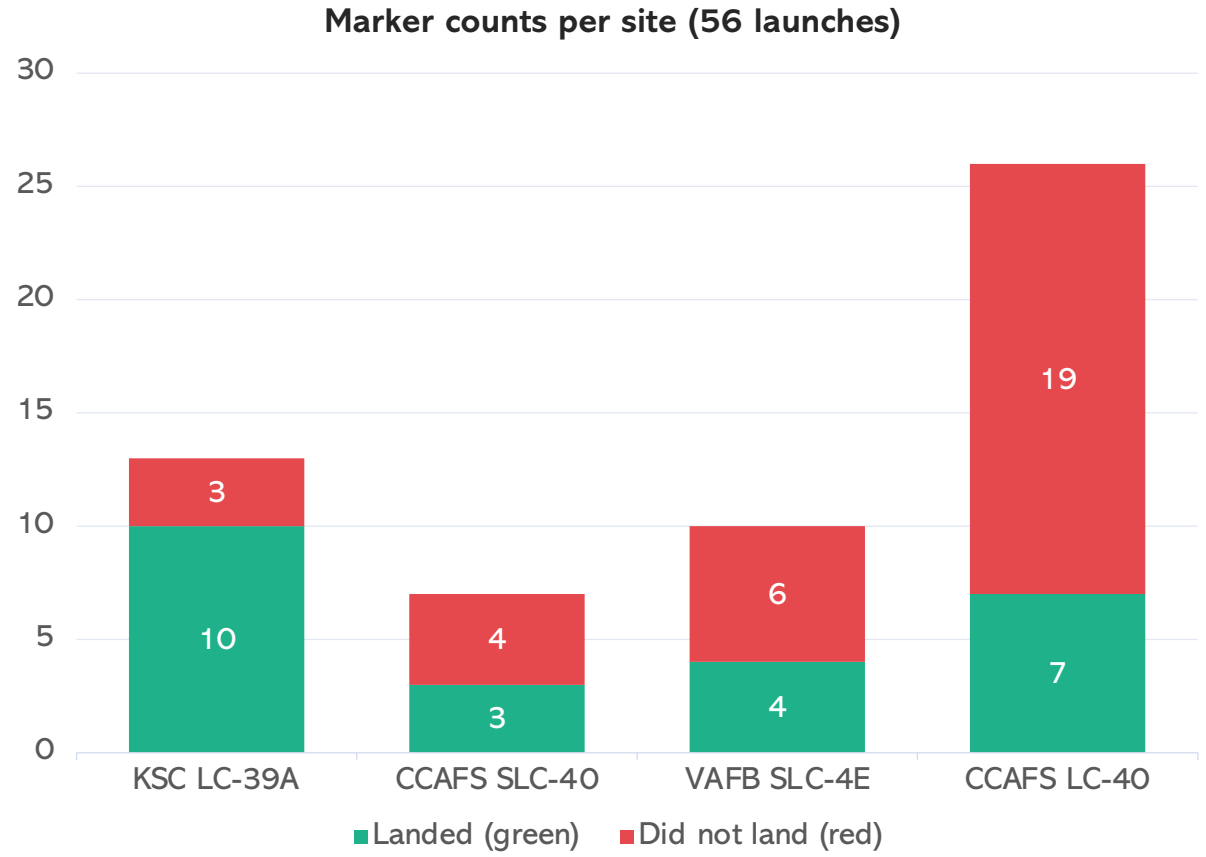
Launch Site Locations



Findings

- 4 launch sites: 3 in Florida (28.6°N) and 1 in California (34.6°N)
- All sit on a coast, so rockets fly over water
- None is on the equator (0°): the sites are between 28°N and 35°N
- On the Folium map each site is a purple circle with its name label

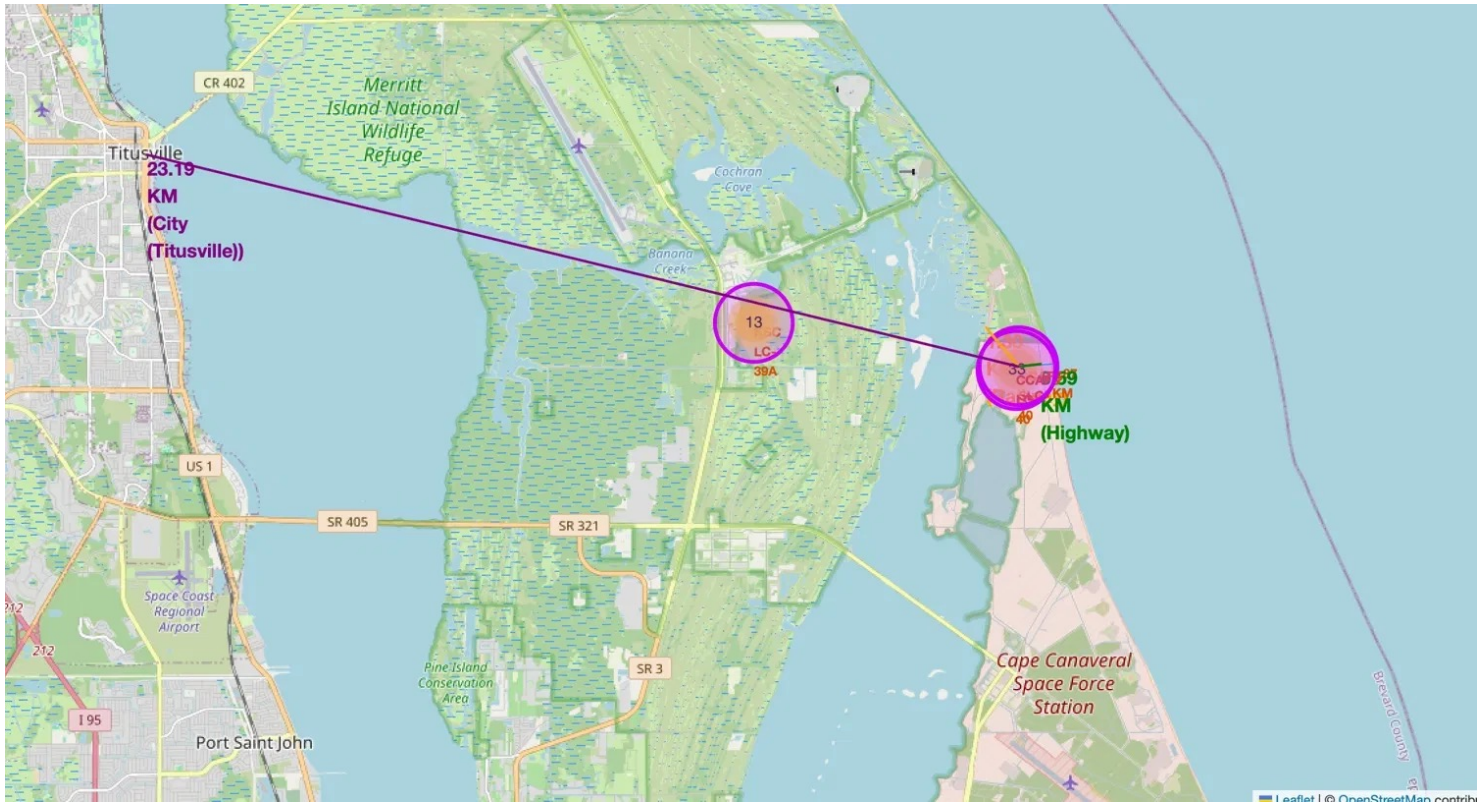
Launch Outcomes by Site (Colour-Labelled Markers)



Findings

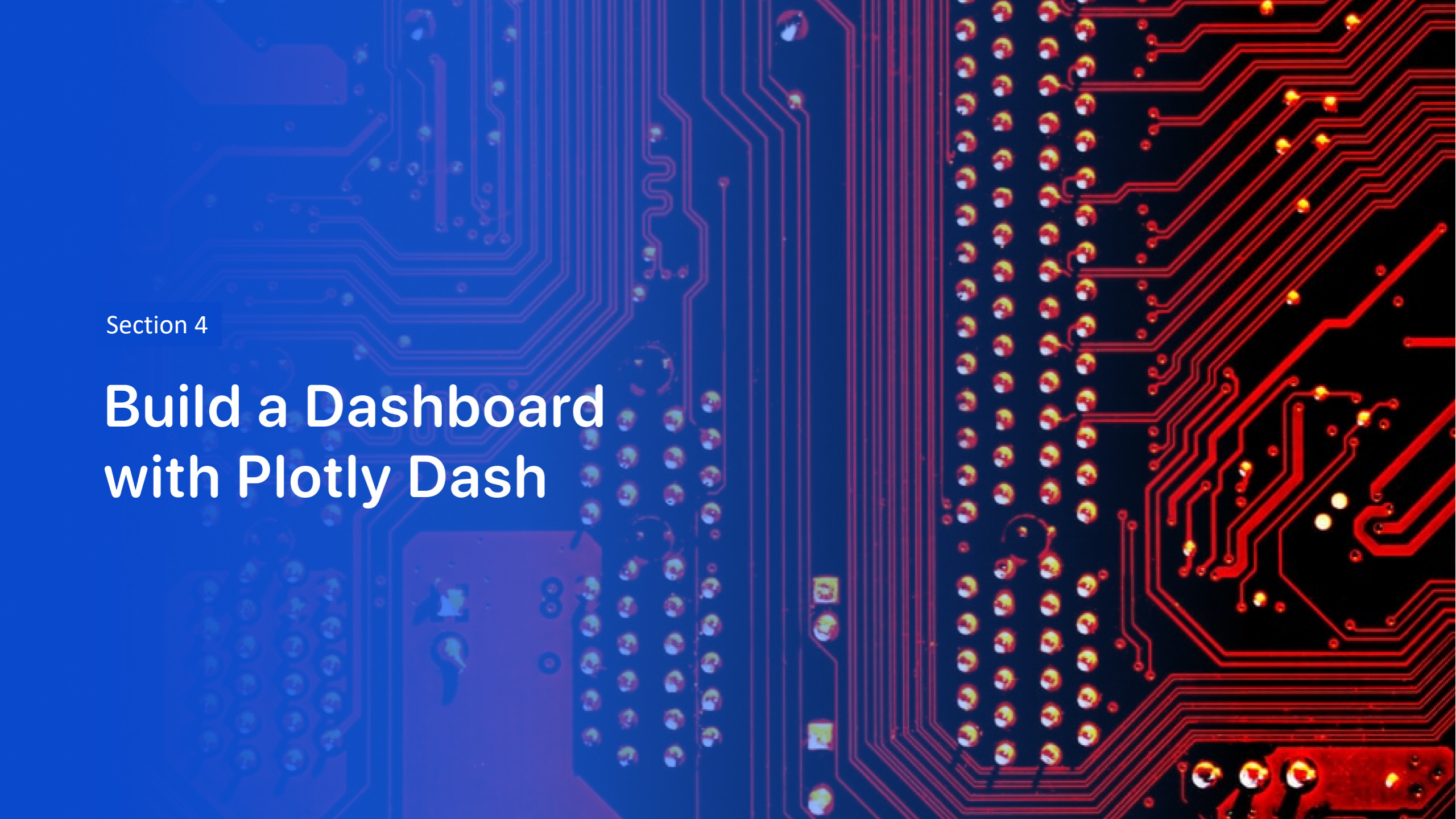
- Every launch is one marker on the Folium map: green = landed, red = did not land
- KSC LC-39A has the best record: 10 of 13 (77%)
- VAFB SLC-4E: 4 of 10 (40%); CCAFS SLC-40: 3 of 7 (43%)
- CCAFS LC-40 has the most launches (26) and the lowest rate: 7 of 26 (27%), likely because it hosted many early flights

Distances from CCAFS LC-40 to Its Proximities



Findings

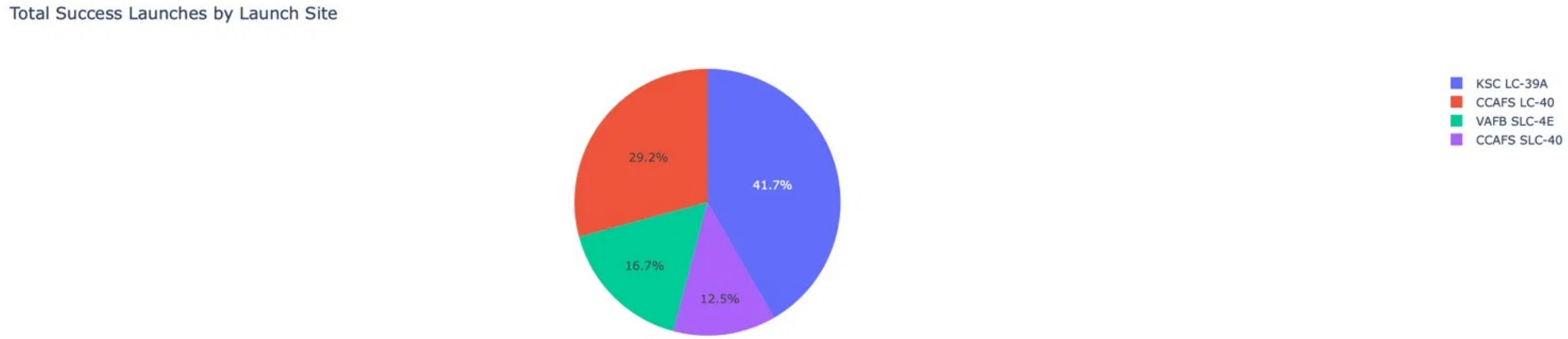
- Purple line: Titusville is 23.19 km away. Cities stay far from the pads, as a safety buffer
- Green line: highway at 0.59 km, easy access for crews and equipment
- Orange line: railway at 1.30 km, heavy stages and engines arrive by rail
- Coastline at 0.87 km: launches go out over open water



Section 4

Build a Dashboard with Plotly Dash

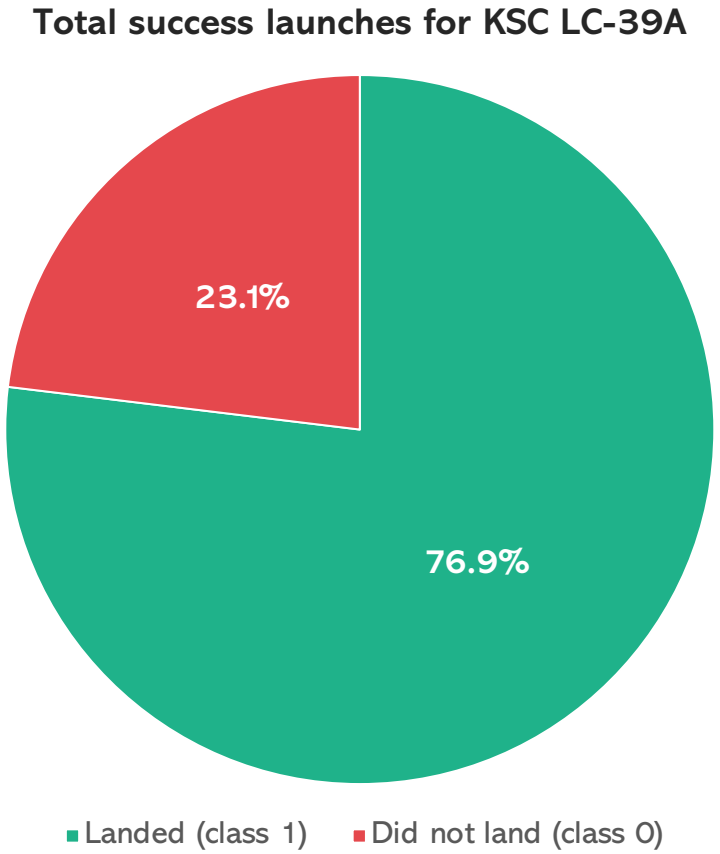
Dashboard: Launch Success Count by Site



Findings

- Dashboard with 'All Sites' selected: the pie shows each site's share of the 24 successful landings
- KSC LC-39A leads with 10 successes (41.7%)
- CCAFS LC-40: 7 (29.2%); VAFB SLC-4E: 4 (16.7%); CCAFS SLC-40: 3 (12.5%)

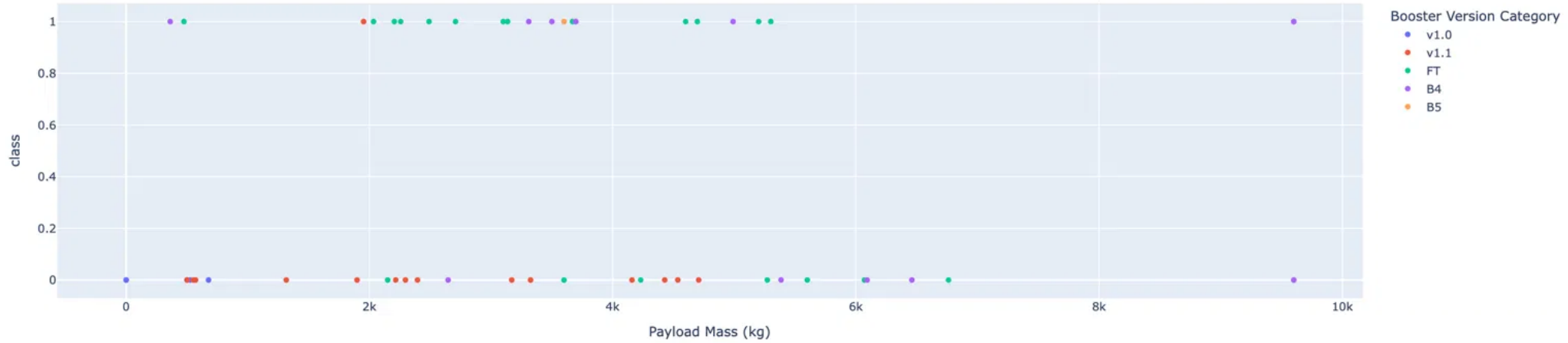
Dashboard: Site with the Highest Success Ratio



Findings

- KSC LC-39A has the highest success ratio: 10 of 13 launches (76.9%)
- Selecting a single site in the dropdown switches the pie to class 1 vs class 0
- Use the other sites to compare: CCAFS LC-40 is the lowest at about 27%

Dashboard: Payload vs. Launch Outcome



Findings

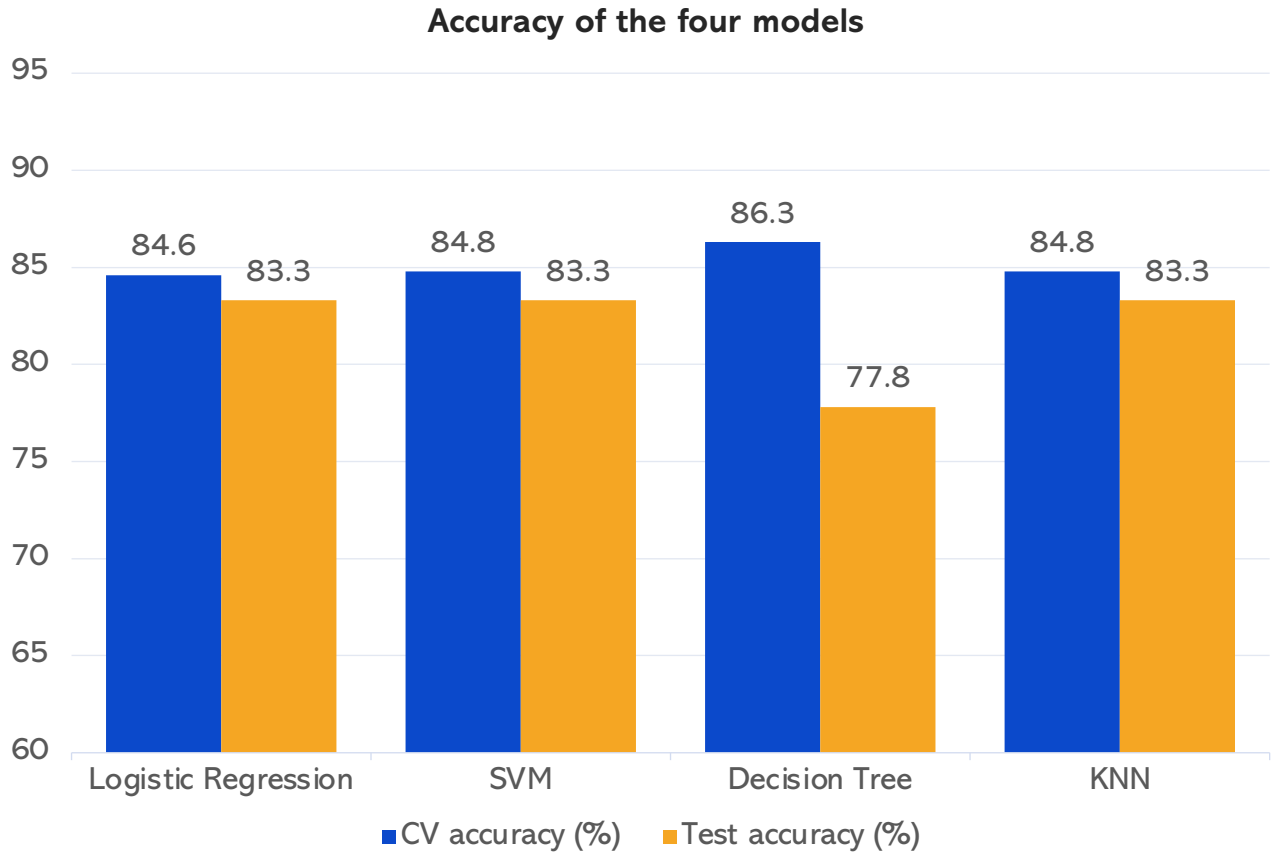
- Each dot is one launch: payload mass (x) against class (y), 1 = landed and 0 = did not land, coloured by booster version
- The old boosters v1.0 and v1.1 mostly sit at class 0, while FT (green) has more landings than failures; the single B5 launch landed
- Payload alone does not decide the outcome: both classes appear across the whole range, even at ~9,600 kg (B4)

Section 5

Predictive Analysis (Classification)



Classification Accuracy

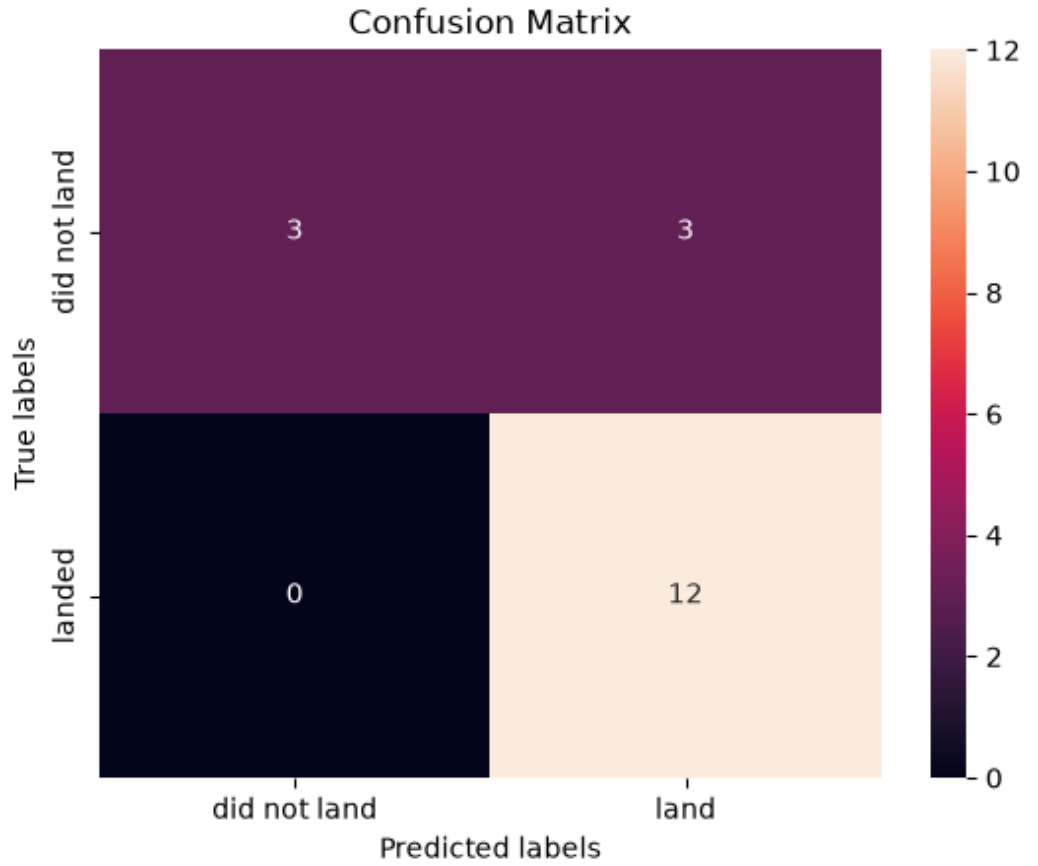


Findings

- Highest test accuracy: Logistic Regression, SVM and KNN tie at 83.3% (15 of 18)
- Decision Tree has the best CV score (86.3%) but the lowest test accuracy (77.8%)
- With only 18 test rows, one prediction moves accuracy by 5.6 points

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_4.ipynb

Confusion Matrix (Logistic Regression)



Explanation

- The three tied models (Logistic Regression, SVM, KNN) have the same matrix
- All 12 real landings were predicted correctly (0 false negatives)
- 3 of the 6 non-landings were wrongly predicted as landings (false positives)
- So the models are optimistic: good at spotting landings, weaker at spotting failures

Notebook: github.com/lucaslopatka/ibm-ds-capstone-spacex/Module_10_4.ipynb

Conclusions

1	**Experience pays off**	Success rate rose from 0% in 2013 to about 85–90% in 2019–2020.
2	**Context matters**	Orbit, payload and launch site relate to the outcome; GTO is the hardest orbit and KSC LC-39A the best site.
3	**Good, not perfect, predictions**	Logistic Regression, SVM and KNN reach 83.3% test accuracy; the tree reaches 77.8%.
4	**Models are optimistic**	They never miss a landing but call 3 of 6 failures a success.
5	**Next steps**	Use more data and a time-based split; try boosted trees; track precision and recall.

Appendix

Code snippets and tuned hyperparameters

Haversine distance (Module_10_3)

```
def calculate_distance(lat1, lon1, lat2, lon2):
    R = 6373.0  # Earth radius in km
    lat1, lon1, lat2, lon2 = map(radians,
                                 [lat1, lon1, lat2, lon2])
    a = (sin((lat2-lat1)/2)**2 +
         cos(lat1)*cos(lat2)*sin((lon2-lon1)/2)**2)
    return R * 2 * atan2(sqrt(a), sqrt(1-a))
```

Model pipeline (Module_10_4)

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=2)
scaler = preprocessing.StandardScaler()
x_train_t = scaler.fit_transform(x_train)
x_test_t = scaler.transform(x_test)
cv = GridSearchCV(model, params, cv=10)
cv.fit(x_train_t, y_train)
```

Best hyperparameters found by GridSearchCV

Model	Best parameters	CV acc.	Test acc.
Logistic Regression	C = 0.01, penalty = l2, solver = lbfgs	84.6%	83.3%
SVM	kernel = sigmoid, C = 1, gamma = 0.032	84.8%	83.3%
Decision Tree	entropy, max_depth = 2, min_samples_split = 5	86.3%	77.8%
KNN	n_neighbors = 10, p = 1, algorithm = auto	84.8%	83.3%

Thank you!

